COLLECTION FRAMEWORK

- The **Collection in Java** is a framework that provides an architecture to store and manipulate the group of objects.

- Java Collections can achieve all the operations that you perform on a data such as searching, sorting, insertion, manipulation, and deletion.

- Java Collection means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, Deque) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).

Collection Framework = set of classes & interfaces used to store and manipulate group of objects in Java. It replaces old style Arrays & Vectors and provides better performance, flexibility, and algorithms.

What is a framework in Java?

- A framework provides a ready-made structure of classes and interfaces for building software applications efficiently.

- This approach enhances object-oriented design, making development quicker, more consistent, and reliable.
  - It provides readymade architecture.
  - It represents a set of classes and interfaces.
  - It is optional.

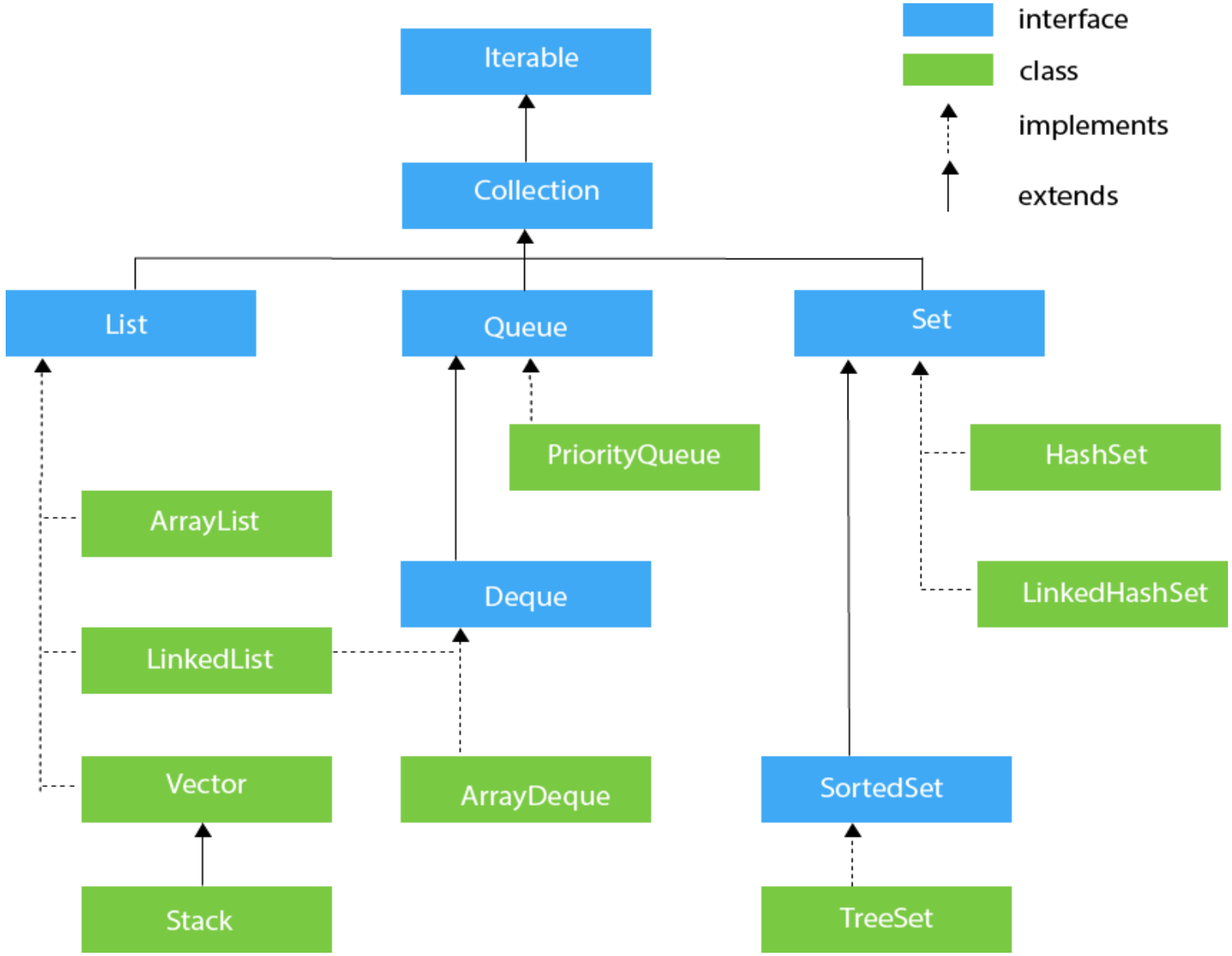
Key Interfaces in Collection Framework

Interface	Description
Collection	Root interface for all collections
List	Ordered collection, allows duplicates
Set	No duplicate elements
Queue	Stores elements in FIFO order
Map	Stores key-value pairs (keys are unique)

Common Implementing Classes

Interface	Common Classes
List	ArrayList, LinkedList, Vector, Stack
Set	HashSet, LinkedHashSet, TreeSet
Queue	PriorityQueue, ArrayDeque
Map	HashMap, LinkedHashMap, TreeMap, Hashtable

HIERARCHY OF COLLECTION FRAMEWORK



Example: Using ArrayList

```
import java.util.*;

class Demo {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Java");
        list.add("Python");
        list.add("C++");

        System.out.println(list);
    }
}
```

- The *collection interfaces* are divided into two groups
  - Java.util.collection
  - Java.util.map

java.util.collection has the following descendants:

- java.util.Set
- java.util.SortedSet
- java.util.NavigableSet
- java.util.Queue
- java.util.concurrent.BlockingQueue
- java.util.concurrent.TransferQueue
- java.util.Deque
- java.util.concurrent.BlockingDeque

java.util.map has the following offspring:

- java.util.SortedMap
- java.util.NavigableMap
- java.util.concurrent.ConcurrentMap
- java.util.concurrent.ConcurrentNavigableMap

Iterable Interface

- The Iterable interface is the root interface for all the collection classes.
- The Collection interface extends the Iterable interface and therefore all the subclasses of Collection interface also implement the Iterable interface.
- It contains only one abstract method. i.e.,

  Iterator<T> iterator()

Collection Interface

- The Collection interface is the interface which is implemented by all the classes in the collection framework.

- Some of the methods of Collection interface are :
  - Boolean add (Object obj),
  - Boolean addAll (Collection c),
  - void clear(), etc.

  That are implemented by all the subclasses of Collection interface.

1. List Interface

- List interface is implemented by the classes ArrayList, LinkedList, Vector, and Stack.

- To instantiate the List interface, we must use:

1. List <data-type> list1= **new** ArrayList();
2. List <data-type> list2 = **new** LinkedList();
3. List <data-type> list3 = **new** Vector();
4. List <data-type> list4 = **new** Stack();

ArrayList

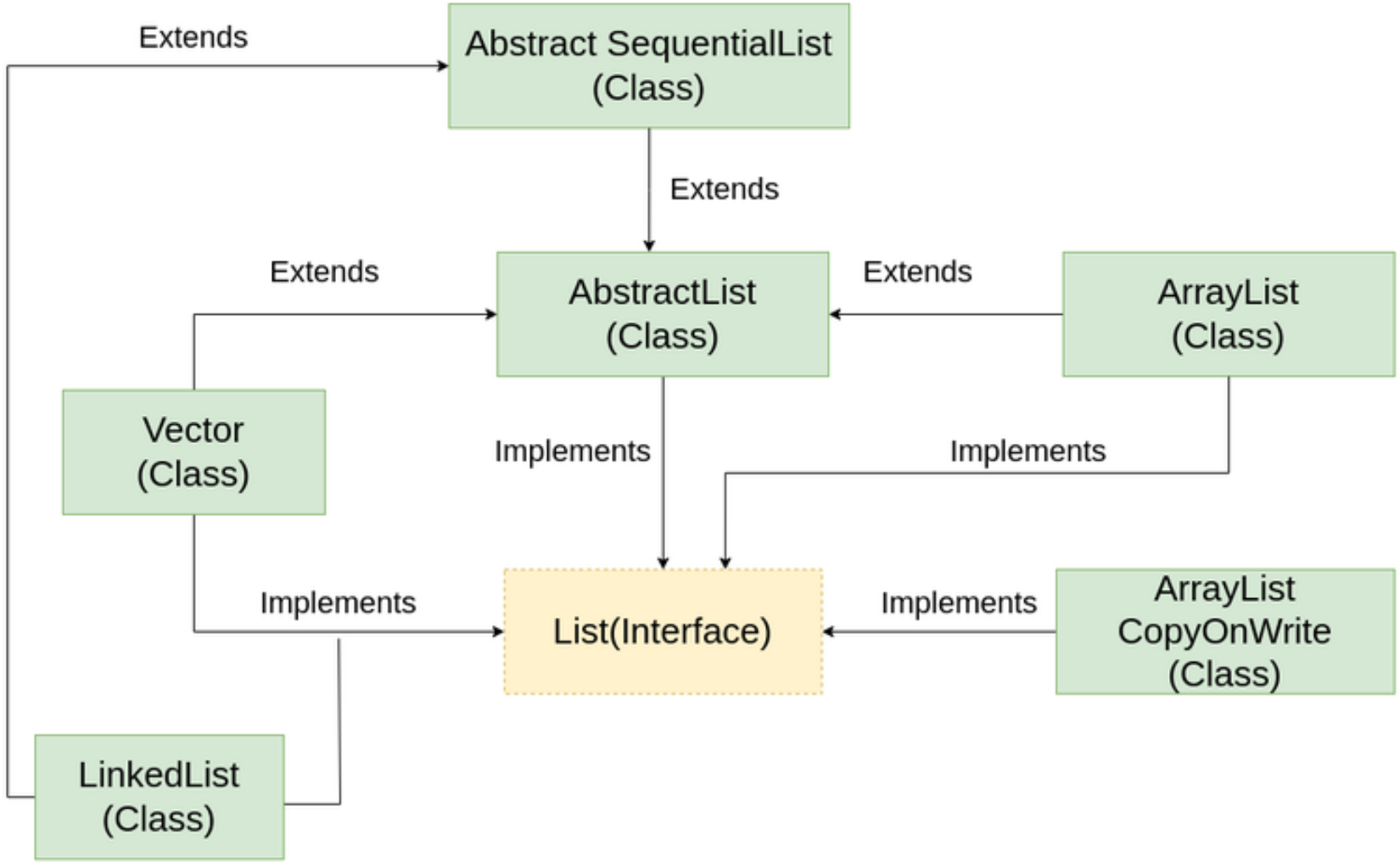
- The ArrayList class implements the List interface. It uses a dynamic array to store the duplicate element of different data types.

- The elements stored in the ArrayList class can be randomly accessed.

Example:

```
import java.util.*;
class TestJavaCollection1{
public static void main(String args[]){
ArrayList<String> list=new ArrayList<String>();//Creating arraylist
list.add("Ravi");//Adding object in arraylist
list.add("Vijay");
list.add("Ravi");
list.add("Ajay");
//Traversing list through Iterator
Iterator itr=list.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
}
}
```

Java ArrayList



- **AbstractList:** This class is used to implement an unmodifiable list, for which one needs to only extend this AbstractList Class and implement only the *get()* and the *size()* methods.

- **CopyOnWriteArrayList:** This class implements the list interface. It is an enhanced version of ArrayList in which all the modifications(add, set, remove, etc.) are implemented by making a fresh copy of the list.

- **AbstractSequentialList:** This class implements the Collection interface and the AbstractCollection class. This class is used to implement an unmodifiable list, for which one needs to only extend this AbstractList Class and implement only the *get()* and the *size()* methods

Array vs ArrayList

Feature	Array	ArrayList
Size	Fixed	Dynamic (Resizable)
Type	Can store primitives & objects	Stores **only objects**
Methods	No inbuilt methods	Has many methods (add, remove, get, etc.)
Performance	Fast	Slight overhead

- Example:

Create an ArrayList object called obj that will store strings:

```
import java.util.ArrayList;                                    //import ArrayList class
ArrayList <String> obj = new ArrayList <String> ();          //create an ArrayList object
```

Add Items

```
import java.util.ArrayList;
public class Main {
public static void main(String[] args)
 {
ArrayList<String> obj = new ArrayList<String>();

 obj.add(“car");
obj.add(“bus");
obj.add(“van");
obj.add(“bike");
System.out.println(obj);

}
}
```

Output:

[car, bus, van, bike]

Adding in specific position

```
import java.util.ArrayList;

public class Main {
public static void main(String[] args)
 {
ArrayList<String> obj = new ArrayList<String>();

obj.add(“car");
obj.add(“bus");
obj.add(“van");
obj.add(“bike");

obj.add(0,“metro");    //Insert element at the beginning of the list (index 0)


System.out.println(obj);

}
}
```

Output:

[metro, car, bus, van, bike]

Access an item

```
import java.util.ArrayList;
public class Main {
public static void main(String[] args)
 {
ArrayList<String> obj = new ArrayList<String>();

obj.add(“car");
obj.add(“bus");
obj.add(“van");
obj.add(“bike");


System.out.println(obj.get(0));

}
}
```

Change an Item

```
import java.util.ArrayList;
public class Main {
public static void main(String[] args)
 {
ArrayList<String> obj = new ArrayList<String>();

obj.add(“car");
obj.add(“bus");
obj.add(“van");
obj.add(“bike");

obj.set (0,“train");

System.out.println(obj);

}
}
```

Output:

[train, bus, van, bike]

Remove an Item

```
import java.util.ArrayList;
public class Main {
public static void main(String[] args)
 {
ArrayList<String> obj = new ArrayList<String>();

obj.add(“car");
obj.add(“bus");
obj.add(“van");
obj.add(“bike");

obj.remove (0);


System.out.println(obj);

}
}
```

Output:

[bus, van, bike]

Clear method

```
import java.util.ArrayList;
public class Main {
public static void main(String[] args)
 {
ArrayList<String> obj = new ArrayList<String>();

obj.add(“car");
obj.add(“bus");
obj.add(“van");
obj.add(“bike");

obj.clear ();


System.out.println(obj);

}
}
```

Size of ArrayList

```
import java.util.ArrayList;
public class Main {
public static void main(String[] args)
 {
ArrayList<String> obj = new ArrayList<String>();

obj.add(“car");
obj.add(“bus");
obj.add(“van");
obj.add(“bike");


System.out.println(obj.size());

}
}
```

Loop Through an ArrayList

```
import java.util.ArrayList;
public class Main {
public static void main(String[] args)
 {
ArrayList<String> obj = new ArrayList<String>();

obj.add(“car");
obj.add(“bus");
obj.add(“van");
obj.add(“bike");


 for (int i = 0;  i < obj.size();  i++) {

            System.out.println(obj.get(i));
}

}

}
```

ArrayList with the for-each loop

```
import java.util.ArrayList;
public class Main {
public static void main(String[] args)
 {
ArrayList<String> obj = new ArrayList<String>();

obj.add(“car");
obj.add(“bus");
obj.add(“van");
obj.add(“bike");

 for (String i : obj) {
            System.out.println(i);
}
 }
}
```

Sort an ArrayList

```
import java.util.ArrayList;
import java.util.Collections;
public class Main {
public static void main(String[] args)
 {
ArrayList<String> obj = new ArrayList<String>();

obj.add(“car");
obj.add(“bus");
obj.add(“van");
obj.add(“bike");

collections.sort(obj);

for (String i : obj) {

        System.out.println(i);
}

}

}
```

Output?

```
import java.util.ArrayList;
public class Main {
    public static void main(String[] args) {
    ArrayList<String> vehicles = new ArrayList<>();
         vehicles.add("Car");
        vehicles.add("Bus");
        vehicles.add("Bike");
        vehicles.add("Van");
         System.out.println("Vehicle List: " + vehicles);
         System.out.println("Element at index 1: " + vehicles.get(1));
        vehicles.set(2, "Scooter");
        System.out.println("After update: " + vehicles);
         vehicles.remove(0);
        System.out.println("After removal: " + vehicles);
        System.out.println("Total vehicles: " + vehicles.size());
    }
}
```

Other Methods of ArrayList Class

Methods	Descriptions
size()	Returns the length of the arraylist.
sort()	Sort the arraylist elements.
clone()	Creates a new arraylist with the same element, size, and capacity.
contains()	Searches the arraylist for the specified element and returns a boolean result.
ensureCapacity()	Specifies the total element the arraylist can contain.
isEmpty()	Checks if the arraylist is empty.
indexOf()	Searches a specified element in an arraylist and returns the index of the element.

Clone method

```
import java.util.ArrayList;
class Main {
    public static void main(String[] args){
        // create an arraylist
        ArrayList<Integer> number = new ArrayList<>();
        number.add(1);
        number.add(3);
        number.add(5);
        System.out.println("ArrayList: " + number);
        // create copy of number
        ArrayList<Integer> cloneNumber = (ArrayList<Integer>)number.clone();
        System.out.println("Cloned ArrayList: " + cloneNumber);
    }
}
```

ArrayList of Custom Objects + Iteration

```
import java.util.ArrayList;
class Student {
    int id;
    String name;
    String course;
    Student(int id, String name, String course) {
        this.id = id;
        this.name = name;
        this.course = course;
    }
}
public class Main {
    public static void main(String[] args) {
        ArrayList<Student> list = new ArrayList<>();
        list.add(new Student(101, “Raj", "Computer Science"));
        list.add(new Student(102, "Amit", "Electronics"));
        list.add(new Student(103, "Priya", "Mechanical"));
        System.out.println("Student Details:");
        for (Student s : list) {
            System.out.println(s.id + " - " + s.name + " - " + s.course);
        }
    }
}
```

Exercise

1. Write a Java program to create an array list, add some colors (strings) and print out the collection.

2. Write a Java program to iterate through all elements in an array list.

3. Write a Java program to insert an element into the array list at the first position.

4. Write a Java program to remove the third element from an array list.

5. Write a Java program to search for an element in an array list. [e.g. contains()]

6. Write a Java program to reverse elements in an array list.

LinkedList

- It uses a doubly linked list internally to store the elements.
- It can store the duplicate elements.
- It maintains the insertion order and is not synchronized.
- In LinkedList, the manipulation is fast because no shifting is required.

How Does LinkedList work Internally?

- Since a LinkedList acts as a dynamic array and we do not have to specify the size while creating it, the size of the list automatically increases when we dynamically add and remove items.

- And also, the elements are not stored in a continuous fashion. Therefore, there is no need to increase the size.

Example

```
import java.util.LinkedList;

public class Main {
  public static void main(String[] args) {
    LinkedList<String> fruits = new LinkedList<String>();
    fruits.add(“apple");
    fruits.add(“orange");
    fruits.add(“grapes");
    fruits.add(“banana");
    System.out.println(fruits);
  }
}
```

addFirst() Adds an item to the beginning of the list

```
import java.util.LinkedList;

public class Main {
  public static void main(String[] args) {
    LinkedList<String> fruits = new LinkedList<String>();
 fruits.add(“apple");
 fruits.add(“banana");
 fruits.add(“orange");

    // Use addFirst() to add the item to the beginning
 fruits.addFirst(“grapes");
    System.out.println(fruits);
  }
}
```

addLast() **Adds an item to the last of the list**

```
import java.util.LinkedList;
public class Main {
  public static void main(String[] args) {
    LinkedList<String> fruits = new LinkedList<String>();
 fruits.add(“apple");
 fruits.add(“banana");
 fruits.add(“orange");

    // Use addLast() to add the item to the beginning
 fruits.addLast(“grapes");
    System.out.println(fruits);
  }
}
```

removeFirst() Remove an item from the beginning of the list

```
import java.util.LinkedList;

public class Main {
  public static void main(String[] args) {
    LinkedList<String> fruits = new LinkedList<String>();
 fruits.add(“apple");
 fruits.add(“banana");
 fruits.add(“orange");

    // Use removeFirst() to add the item to the beginning
 fruits. removeFirst();
    System.out.println(fruits);
  }
}
```

removeLast() Remove an item from the end of the list

```
import java.util.LinkedList;
public class Main {
  public static void main(String[] args) {
    LinkedList<String> fruits = new LinkedList<String>();
 fruits.add(“apple");
 fruits.add(“banana");
 fruits.add(“orange");

    // Use removeLast() to delete the item at the last
 fruits. removeLast();
    System.out.println(fruits);
  }
}
```

the **clone()** method, creates a **shallow copy**

```
// Java program to Demonstrate the  working of clone() in LinkedList

import java.util.LinkedList;
public class Example {
    public static void main(String args[]) {
        LinkedList<String> l = new LinkedList<>();
        // Use add() method to add  elements in the LinkedList
        l.add(“Hello");
        l.add(“World");
        System.out.println("Original LinkedList: " + l);
        // create another list  and copy the elements
        LinkedList l2 = new LinkedList();
        l2 = (LinkedList)l.clone();
        System.out.println("Clone LinkedList is: " + l2);
    }
}
```

descendingIterator() return an iterator that allows to traverse the list in reverse order.

```
// Java program to use  descendingIterator()
import java.util.Iterator;
import java.util.LinkedList;
public class Example {
    public static void main(String[] args) {
            // Creating a LinkedList
        LinkedList<String> l = new LinkedList<>();
        // Adding elements to the LinkedList
        l.add("Java");
        l.add("Python");
        l.add("C++");
        // Display the original LinkedList
        System.out.println("Original list: " + l);
        // Using descendingIterator to  iterate the list in reverse order
        Iterator<String> it = l.descendingIterator();
        // Iterating through the list in reverse order
        System.out.print("Reverse Order list: ");
}
    }
}
```

Exercise

- Music Playlist Manager:

Write a program that allows user to create and manage a music playlist using a Linkedlist. Users can add songs, remove songs, play the playlist, sort the playlist by title and shuffle the playlist.

Exercise

1. Write a program to remove an element from a LinkedList by its index and by its value. Demonstrate the use of remove(index) and remove(Object).
2. Write a program that sorts a LinkedList of integers in ascending order using Collections.sort().
3. Write a program that clones a LinkedList using the clone() method and prints both the original and the cloned list.
4. Write a program that merges two LinkedLists into a single LinkedList by appending elements from the second list to the first.
5. Write a program that uses the forEach() method to iterate over the elements of a LinkedList and prints them.
6. Write a program that removes all elements from the LinkedList that are greater than a certain value.
7. Write a program to check if a LinkedList is empty using isEmpty() method.

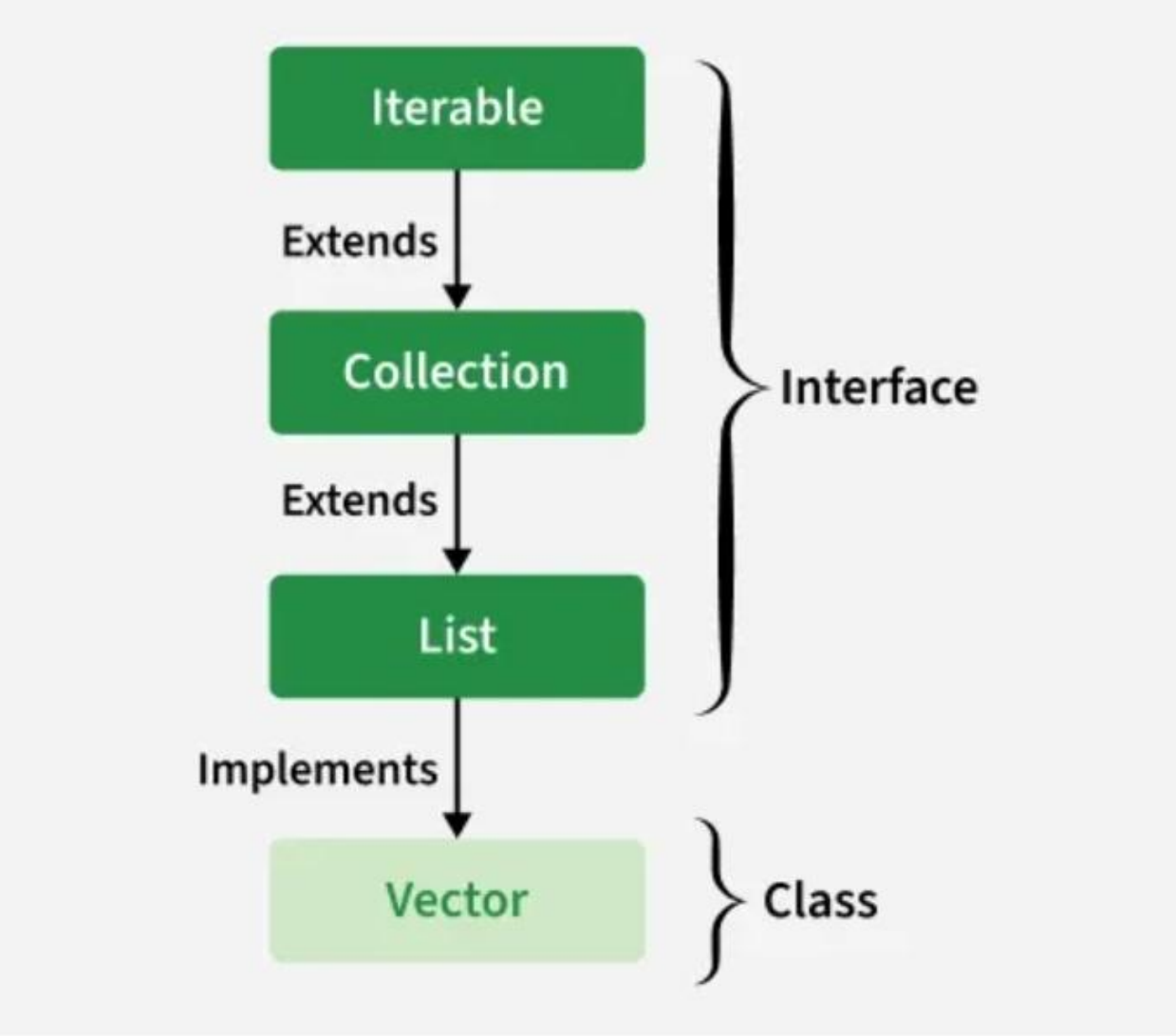
Vector

- Vector uses a dynamic array to store the data elements. It is similar to ArrayList.
- However, it is synchronized and contains many methods that are not the part of Collection framework.
- Implements List, RandomAccess, Cloneable, and Serializable interfaces.
- Vector is a Legacy class that was introduced in early versions of Java.
- Thread-safe: All methods are synchronized for safe multi-threaded access.

Example

```
import java.util.*;
public class TestJavaCollection3{
public static void main(String args[]){
Vector<String> v=new Vector<String>();
v.add("Ayush");
v.add("Amit");
v.add("Ashish");
v.add("Garima");
Iterator<String> itr=v.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
}
}
```

Hierarchy of Vector



```
import java.util.Vector;
public class VectorDoublingExample {
    public static void main(String[] args) {
        Vector<Integer> vector = new Vector<>(2); // initial capacity = 2
        System.out.println("Initial capacity: " + vector.capacity());
        // Add elements to trigger capacity increase
        vector.add(10);
        vector.add(20);
        System.out.println("Capacity after adding 2 elements: " + vector.capacity());
        vector.add(30); // Triggers resize (2 → 4)
        System.out.println("Capacity after adding 3rd element: " + vector.capacity());
        vector.add(40);
        vector.add(50); // Triggers resize again (4 → 8)
        System.out.println("Capacity after adding 5 elements: " + vector.capacity());
    }
}
```

Different Operations of Vector Class

- **add(Object)**: This method is used to add an element at the end of the Vector.
- **add(int index, Object):** This method is used to add an element at a specific index in the Vector.
- **set() :** updating element.
- **remove(Object):** Removes the first occurrence of the specified object.
- **remove(int index):** Removes the element at the given index and shifts remaining elements to the left.
- **get():** method to get the element at a specific index and the advanced for a loop.

```
import java.util.*;
public class Example {
  public static void main(String args[])
    {
       Vector<String> v = new Vector<>();
       v.add(“Apple");
       v.add(“Mango");
       v.add(1, “Grapes");
       for (int i = 0; i < v.size(); i++) {
          System.out.print(v.get(i) + " ");
       }
       System.out.println();
          for (String str : v)
          System.out.print(str + " ");
    }
}
```

HashSet in Java

HashSet in Java implements the Set interface of the Collections Framework. It is used to store the unique elements, and it doesn't maintain any specific order of elements.

- HashSet does not allow duplicate elements.
- Uses HashMap internally which is an implementation of hash table data structure.
- Also implements Serializable and Cloneable interfaces.
- HashSet is not thread-safe. To make it thread-safe, synchronization is needed externally.

Adding Elements in HashSet

```
import java.util.*;
class Example
{
    public static void main(String[] args)
    {
        // Creating an empty HashSet of string entities
        HashSet<String> hs = new HashSet<String>();
        // Adding elements using add() method
        hs.add(“Apple");
        hs.add(“Mango");
        hs.add(“Banana");

        System.out.println("HashSet : " + hs);
    }
}
```

Removing Elements in HashSet

```
import java.util.*;
class Example
{  public static void main(String[] args)
    {
       HashSet<String> hs = new HashSet<String>();
      hs.add("A");
       hs.add("B");
       hs.add("Z");
       System.out.println("HashSet : " + hs);
       // Removing the element B
       hs.remove("B");
       // Printing the updated HashSet elements
       System.out.println("HashSet after removing element : " + hs);
       // Returns false if the element is not present
       System.out.println("B exists in Set : " + hs.remove("B"));
    }
}
```

Methods of HashSet

Method	Description
add(E e)	Used to add the specified element if it is not present, if it is present then return false.
clear()	Used to remove all the elements from the set.
contains(Object o)	Used to return true if an element is present in a set.
remove(Object o)	Used to remove the element if it is present in set.
iterator()	Used to return an iterator over the element in the set.
isEmpty()	Used to check whether the set is empty or not. Returns true for empty and false for a non-empty condition for set.
size()	Used to return the size of the set.
clone()	Used to create a shallow copy of the set.

Stack

- The stack is the subclass of Vector.

- It implements the last-in-first-out data structure, i.e., Stack.

- The stack contains all of the methods of Vector class and also provides its methods like
  - boolean push(),
  - boolean peek(),
  - boolean push(object o)

Example

```
import java.util.*;
public class TestJavaCollection4{
public static void main(String args[]){
Stack<String> stack = new Stack<String>();
stack.push("Ayush");
stack.push("Garvit");
stack.push("Amit");
stack.push("Ashish");
stack.push("Garima");
stack.pop();
Iterator<String> itr=stack.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
}
}
```

2. Queue Interface

- Queue interface maintains the first-in-first-out order.
- It can be defined as an ordered list that is used to hold the elements which are about to be processed.
- There are various classes like PriorityQueue, Deque, and ArrayDeque which implements the Queue interface.

- Queue interface can be instantiated as:

```
Queue<String> q1 = new PriorityQueue();
Queue<String> q2 = new ArrayDeque();
```

PriorityQueue

- The PriorityQueue class implements the Queue interface.
- It holds the elements or objects which are to be processed by their priorities.
- PriorityQueue doesn't allow null values to be stored in the queue.

```
import java.util.*;
public class TestJavaCollection5{
public static void main(String args[]){
PriorityQueue<String> queue=new PriorityQueue<String>();
queue.add("Amit Sharma");
queue.add("Vijay Raj");
queue.add("JaiShankar");
queue.add("Raj");
System.out.println("head:"+queue.element());
System.out.println("head:"+queue.peek());
System.out.println("iterating the queue elements:");
Iterator itr=queue.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
queue.remove();
queue.poll();
System.out.println("after removing two elements:");
Iterator<String> itr2=queue.iterator();
while(itr2.hasNext()){
System.out.println(itr2.next());
}
} }
```

Deque Interface

- In Deque, we can remove and add the elements from both the side.
- Deque stands for a double-ended queue which enables us to perform the operations at both the ends.
- Deque can be instantiated as:

```
Deque d = new ArrayDeque();
```

Example

```
import java.util.*;
public class TestJavaCollection6{
public static void main(String[] args) {
//Creating Deque and adding elements
Deque<String> deque = new ArrayDeque<String>();
deque.add("Gautam");
deque.add("Karan");
deque.add("Ajay");
//Traversing elements
for (String str : deque) {
System.out.println(str);
}
}
}
```

3. Set Interface

- Set Interface in Java is present in java.util package. It extends the Collection interface.
- It represents the unordered set of elements which doesn't allow us to store the duplicate items.
- We can store at most one null value in Set.
- Set is implemented by HashSet, LinkedHashSet, and TreeSet.

- Set can be instantiated as:

1. Set<data-type> s1 = **new** HashSet<data-type>();
2. Set<data-type> s2 = **new** LinkedHashSet<data-type>();
3. Set<data-type> s3 = **new** TreeSet<data-type>();

HashSet

- HashSet class implements Set Interface.
- It represents the collection that uses a hash table for storage.
- Hashing is used to store the elements in the HashSet. It contains unique items.

Example

```
import java.util.*;
public class TestJavaCollection7{
public static void main(String args[]){
//Creating HashSet and adding elements
HashSet<String> set=new HashSet<String>();
set.add("Ravi");
set.add("Vijay");
set.add("Ravi");
set.add("Ajay");
//Traversing elements
Iterator<String> itr=set.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
}
}
```